Java String - Interview Notes

Core Java | Senior Java Backend Interview Revision | Concise explanations + examples + e-commerce use cases

Quick memory: String = immutable | String Pool = reuse literals | == = reference | equals() = content | intern() = pooled reference | StringBuilder = mutable | StringBuffer = mutable + synchronized

1. What is String?

String is a Java class used to represent a sequence of characters. String objects are **immutable**, meaning the value of an existing String object cannot be changed after creation.

```
String s = "Java";
s.concat(" Developer");
System.out.println(s); // Java

s = s.concat(" Developer");
System.out.println(s); // Java Developer
```

E-commerce example: Email, order status text, product name, request parameters and IDs are commonly represented as Strings.

Interview answer: "String is a class in Java used to represent a sequence of characters. String objects are immutable, meaning once a String object is created, its value cannot be changed."

2. Why is String Immutable?

Any operation that appears to modify a String produces a new result instead of changing the original String. Main interview reasons: **security, String Pool reuse, thread safety by immutability, and stable hash-code behavior** when Strings are used as HashMap keys.

```
String s = "Java";
String result = s.concat(" Developer");

System.out.println(s); // Java
System.out.println(result); // Java Developer
```

E-commerce example: Strings are frequently used for URLs, usernames, identifiers, file paths and HashMap keys. Immutability makes shared String values predictable.

Interview answer: "String is immutable mainly for security, String Pool optimization, thread safety, and stable hash-code behavior. Since it cannot change after creation, it can be safely shared and reused."

3. How to Create an Immutable Class

Typical rules: make the class final, fields private and final, initialize them through the constructor, provide no setters, and use defensive copies for mutable fields.

```
public final class Customer {
private final int id;
private final String name;

public Customer(int id, String name) {
this.id = id;
this.name = name;
}

public int getId() { return id; }
```

```
public String getName() { return name; }
}
```

For mutable fields such as List, copy them when accepting and returning them. Modern Java can also use List.copyOf(...) when appropriate.

> **Interview answer:** "I make the class final, fields private and final, initialize them through the constructor, avoid setters, and use defensive copies for mutable objects."

4. String Pool (SCP)

The String Pool is a special pool in the Java heap where String literals are stored so identical literal values can be reused rather than duplicated.

```
String s1 = "Java";
String s2 = "Java";

System.out.println(s1 == s2); // true
System.out.println(s1.equals(s2)); // true
```

Important: String literals are interned/reused from the pool. Using new String(...) explicitly creates a separate String object.

> **Interview answer:** "String Pool is a special area in heap memory where Java stores String literals so identical literals can be reused."

5. new String()

Using **new String(...)** explicitly creates a separate ordinary String object. In modern Java, both ordinary objects and the String Pool are in heap memory, so distinguish the pooled instance from the separate object created with new.

```
String s1 = "Java";
String s2 = new String("Java");

System.out.println(s1 == s2); // false
System.out.println(s1.equals(s2)); // true
```

If the literal "Java" was not already pooled, processing the literal establishes the pooled value and new creates the separate object.

> **Interview point:** With String s = new String("Java"), assuming the literal was not already pooled, two String objects are involved: the pooled literal and the separate object created by new.

6. == vs equals()

For Strings, **== compares references**, while **equals() compares content**.

```
String s1 = "Java";
String s2 = new String("Java");

s1 == s2; // false
s1.equals(s2); // true
```

E-commerce example: Avoid `status == "CONFIRMED"`. Prefer `"CONFIRMED".equals(status)`. For fixed domain states, an enum such as OrderStatus.CONFIRMED is usually even better.

> **Interview answer:** "`==` compares object references, whereas `equals()` compares the content of String objects."

7. intern()

intern() returns the canonical pooled String reference for the same content. Think: "Give me the String Pool reference for this value." It does not mutate the original reference.

```
String s1 = "Java";
String s2 = new String("Java");
String s3 = s2.intern();

System.out.println(s1 == s2); // false
System.out.println(s1 == s3); // true
```

Calling only s2.intern(); does not make s2 point to the pool. Assign the returned reference if you need it: s2 = s2.intern();

> **Interview answer:** "`intern()` returns the canonical String reference from the String Pool for the same content."

8. String vs StringBuilder vs StringBuffer

String is immutable. StringBuilder and StringBuffer are mutable. StringBuilder is not synchronized and is generally preferred for repeated manipulation in normal single-threaded code. StringBuffer uses synchronized methods.

```
StringBuilder message = new StringBuilder();
message.append("Order ID: ").append(orderId);
message.append(", Status: ").append(status);
String finalMessage = message.toString();
```

E-commerce example: Use StringBuilder when constructing an order summary, log text, CSV-like output, or notification message through many append operations.

> **Interview answer:** "String is immutable. StringBuilder and StringBuffer are mutable. StringBuilder is generally faster and not synchronized, whereas StringBuffer uses synchronized methods."

9. final vs Immutable

They are not the same. **final** prevents a variable/reference from being reassigned. **Immutable** means the object's state cannot change after creation.

```
final StringBuilder sb = new StringBuilder("Java");
sb.append(" Developer"); // allowed

// sb = new StringBuilder("Python"); // not allowed
```

A final reference can still point to a mutable object. String itself is immutable; StringBuilder is mutable.

> **Interview answer:** "`final` prevents reference reassignment, whereas immutability prevents the object's internal state from changing."

10. String Concatenation

Simple + concatenation is fine. Compile-time constant expressions can be folded by the compiler. For repeated runtime concatenation, especially inside loops, StringBuilder is usually preferable because String is immutable.

```
String s1 = "Java" + "Developer";
String s2 = "JavaDeveloper";
System.out.println(s1 == s2); // true

StringBuilder result = new StringBuilder();
```

```
for (int i = 0; i < 1000; i++) {
result.append(i);
}
```

Historically, runtime concatenation was often explained as StringBuilder-based. Modern Java can use more advanced compiler/JVM concatenation mechanisms, so avoid saying every + literally creates a StringBuilder.

> **Interview answer:** "Since String is immutable, concatenation produces a new result. Compile-time constants may be optimized, and for repeated runtime concatenation I prefer StringBuilder."

11. Common String Methods

Method	Example	Meaning / Result
length()	s.length()	number of characters
charAt()	s.charAt(0)	character at index
substring()	s.substring(5)	part of a String
contains()	s.contains("Java")	checks contained text
startsWith()	s.startsWith("Java")	checks prefix
endsWith()	s.endsWith("Developer")	checks suffix
replace()	s.replace("Java","Spring")	returns replaced result
indexOf()	s.indexOf("Developer")	starting index
split()	s.split(",")	splits into array
isEmpty()	"".isEmpty()	true when length is 0
isBlank()	" ".isBlank()	true for empty/whitespace
strip()	s.strip()	Unicode-aware whitespace removal

isEmpty() vs isBlank(): " ".isEmpty() is false, but " ".isBlank() is true.

Real-time example: `String normalizedEmail = email.strip().toLowerCase();`

12. String Object-Creation Questions

Example: String s1 = "Java"; String s2 = "Java"; String s3 = new String("Java");

Assuming "Java" did not exist in the pool before these statements, two String objects are created: one pooled "Java" reused by s1 and s2, and one separate object created by new for s3.

Interview tip: When asked "How many objects?", qualify the answer with: *"Assuming the literal was not already present in the String Pool..."*

13. Important Interview Q&A;

Q1. What is String in Java?
String is a class representing a sequence of characters, and String objects are immutable.

Q2. Why is String immutable?
For reasons including security, safe pool reuse, thread safety by immutability, and stable hash-code behavior.

Q3. Where is the String Pool?
In modern Java, the String Pool is in heap memory.

Q4. Does JVM always check SCP when creating a String?
No. String literals are interned/reused from the pool, while new String(...) explicitly creates a separate object.

Q5. == vs equals()?
== compares references; equals() compares content.

Q6. What does intern() do?
Returns the canonical pooled reference for equal String content.

Q7. StringBuilder vs StringBuffer?
Both are mutable; StringBuilder is not synchronized, while StringBuffer uses synchronized methods.

Q8. Why use StringBuilder in loops?
To avoid inefficient repeated creation of intermediate String values during many concatenations.

Q9. final vs immutable?
final prevents reassignment; immutable means object state cannot change.

Q10. How do you make a custom immutable class?
final class, private final fields, constructor initialization, no setters, and defensive copies for mutable fields.

Q11. Why is String a good HashMap key?
Its immutability makes its value/hash behavior stable after creation.

Q12. isEmpty() vs isBlank()?
isEmpty checks length == 0; isBlank also treats whitespace-only content as blank.

Q13. trim() vs strip()?
Both remove surrounding whitespace in common cases; strip() is Unicode-aware.

Q14. Can a final StringBuilder change?
Yes. The reference cannot be reassigned, but the mutable StringBuilder object can be modified.

Q15. Does intern() change the original variable?
No. intern() returns a reference; assign the return value if you want to use the pooled reference.

14. Quick Revision Sheet

Concept	Remember
String	Immutable
String Pool	Reuses interned/literal values
==	Reference comparison
equals()	Content comparison
intern()	Canonical pooled reference
StringBuilder	Mutable, not synchronized; preferred for repeated manipulation
StringBuffer	Mutable; synchronized methods
final	Reference/variable cannot be reassigned
Immutable	Object state cannot change
isEmpty()	Length is zero
isBlank()	Empty or whitespace only

Final interview memory: String is immutable; literals can be pooled and reused; use equals() for content; use StringBuilder for repeated modifications; understand intern(), object creation, and final vs immutable.